

PANDAS PROGRAM

1. Student Marks

Question:

Given the following student DataFrame, calculate the Total and Average marks for each student and display students whose average is greater than 75.

Input:

Name	Python	Java	DBMS
Arun	80	85	75
Bala	65	70	60
Charan	90	88	92
Divya	75	80	78

Expected Output:

Name	Total	Average
Arun	240	80.00
Bala	195	65.00
Charan	270	90.00
Divya	233	77.67

Students with Average > 75:

Name	Average
Arun	80.00
Charan	90.00
Divya	77.67

The screenshot shows a Jupyter Notebook interface with a code editor on the left and an output area on the right. The code in the editor defines a DataFrame with student names and marks in Python, Java, and DBMS. It then calculates the total marks for each student and the average marks. Finally, it filters the DataFrame to show only students with an average mark greater than 75.

```
main.py Visualize Run
```

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Arun", "Bala", "Charan", "Divya"],
5     "Python": [80, 65, 90, 75],
6     "Java": [85, 70, 88, 80],
7     "DBMS": [75, 60, 92, 78]
8 }
9
10 df = pd.DataFrame(data)
11
12 df["Total"] = df["Python"] + df["Java"] + df["DBMS"]
13 df["Average"] = df["Total"] / 3
14
15 print(df[["Name", "Total", "Average"]])
16
17 print("\nStudents with Average > 75:")
18 print(df.loc[df["Average"] > 75, ["Name", "Average"]])
```

Output

	Name	Total	Average
0	Arun	240	80.000000
1	Bala	195	65.000000
2	Charan	270	90.000000
3	Divya	233	77.666667

Students with Average > 75:

	Name	Average
0	Arun	80.000000
2	Charan	90.000000
3	Divya	77.666667

=== Code Execution Successful ===

2. Employee Salary

Question:

Given employee details, find the highest-paid employee in each department.

Input:

ID	Name	Department	Salary
101	Ravi	IT	50000
102	Kumar	HR	45000
103	Anu	IT	65000
104	Priya	HR	55000
105	Manoj	Sales	60000

Expected Output:

Department	Name	Salary
HR	Priya	55000
IT	Anu	65000
Sales	Manoj	60000

```
main.py Visualize Run Output
1 import pandas as pd
2
3 data = {
4     "ID": [101, 102, 103, 104, 105],
5     "Name": ["Ravi", "Kumar", "Anu", "Priya", "Manoj"],
6     "Department": ["IT", "HR", "IT", "HR", "Sales"],
7     "Salary": [50000, 45000, 65000, 55000, 60000]
8 }
9
10 df = pd.DataFrame(data)
11
12 highest = df.loc[df.groupby("Department")["Salary"].idxmax()]
13
14 print(highest[["Department", "Name", "Salary"]].sort_values
      ("Department"))

Department  Name  Salary
3          HR  Priya  55000
2           IT   Anu  65000
4          Sales Manoj  60000

=== Code Execution Successful ===
```

3. Product Sales

Question:

Given product details, calculate $\text{Total_Sales} = \text{Quantity} \times \text{Price}$. Find the total sales for each category.

Input:

Product	Category	Quantity	Price
Laptop	Electronics	5	50000
Mouse	Accessories	20	500
Keyboard	Accessories	10	1000
Mobile	Electronics	8	20000
Tablet	Electronics	6	15000

Expected Output:

Product	Total_Sales
Laptop	250000
Mouse	10000
Keyboard	10000
Mobile	160000
Tablet	90000

Category	Total_Sales
Accessories	20000
Electronics	500000

main.py

Visualize

Run

```
1 import pandas as pd
2
3 data = {
4     "Product": ["Laptop", "Mouse", "Keyboard", "Mobile", "Tablet"],
5     "Category": ["Electronics", "Accessories", "Accessories", "Electronics", "Electronics"],
6     "Quantity": [5, 20, 10, 8, 6],
7     "Price": [50000, 500, 1000, 20000, 15000]
8 }
9
10 df = pd.DataFrame(data)
11
12 df["Total_Sales"] = df["Quantity"] * df["Price"]
13
14 print(df[["Product", "Total_Sales"]])
15
16 print("\nCategory-wise Total Sales:")
17 print(df.groupby("Category")["Total_Sales"].sum())
```

Output

	Product	Total_Sales
0	Laptop	250000
1	Mouse	10000
2	Keyboard	10000
3	Mobile	160000
4	Tablet	90000

Category-wise Total Sales:

Category	Total_Sales
Accessories	20000
Electronics	500000

Name: Total_Sales, dtype: int64

=== Code Execution Successful ===

4. Student Attendance

Question:

Given student attendance details, calculate the attendance percentage and display students whose attendance is below 80%.

Input:

Name	Total_Days	Present_Days
Arun	50	45
Bala	50	38
Charan	50	48
Divya	50	35

Expected Output:

Name	Attendance %
Arun	90.0
Bala	76.0
Charan	96.0
Divya	70.0

Students below 80%:

Name	Attendance %
Bala	76.0
Divya	70.0

main.py

Visualize

Run

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Arun", "Bala", "Charan", "Divya"],
5     "Total_Days": [50, 50, 50, 50],
6     "Present_Days": [45, 38, 48, 35]
7 }
8
9 df = pd.DataFrame(data)
10
11 df["Attendance %"] = (df["Present_Days"] / df["Total_Days"]) * 100
12
13 print(df[["Name", "Attendance %"]])
14
15 print("\nStudents below 80%:")
16 print(df.loc[df["Attendance %"] < 80, ["Name", "Attendance %"]])
```

Output

	Name	Attendance %
0	Arun	90.0
1	Bala	76.0
2	Charan	96.0
3	Divya	70.0

Students below 80%:

	Name	Attendance %
1	Bala	76.0
3	Divya	70.0

=== Code Execution Successful ===

5. Product Rating Analysis

Question:

Given product ratings, find the average rating for each category and display the category having the highest average rating.

Input:

Product	Category	Rating
Laptop	Electronics	4.5
Mobile	Electronics	4.2
Mouse	Accessories	3.8
Keyboard	Accessories	4.6
Tablet	Electronics	4.8

Expected Output:

Category	Average Rating
Accessories	4.20
Electronics	4.50

Highest Rated Category:

Electronics 4.50

main.py

Visualize

Run

```
1 import pandas as pd
2
3 data = {
4     "Product": ["Laptop", "Mobile", "Mouse", "Keyboard", "Tablet"],
5     "Category": ["Electronics", "Electronics", "Accessories",
6     "Accessories", "Electronics"],
7     "Rating": [4.5, 4.2, 3.8, 4.6, 4.8]
8 }
9
10 df = pd.DataFrame(data)
11
12 average_rating = df.groupby("Category")["Rating"].mean()
13
14 print("Category-wise Average Rating:")
15 print(average_rating)
16
17 highest = average_rating.idxmax()
18 highest_rating = average_rating.max()
19
20 print("\nHighest Rated Category:")
21 print(highest, highest_rating)
```

Output

Category-wise Average Rating:
Category
Accessories 4.2
Electronics 4.5
Name: Rating, dtype: float64

Highest Rated Category:
Electronics 4.5

=== Code Execution Successful ===